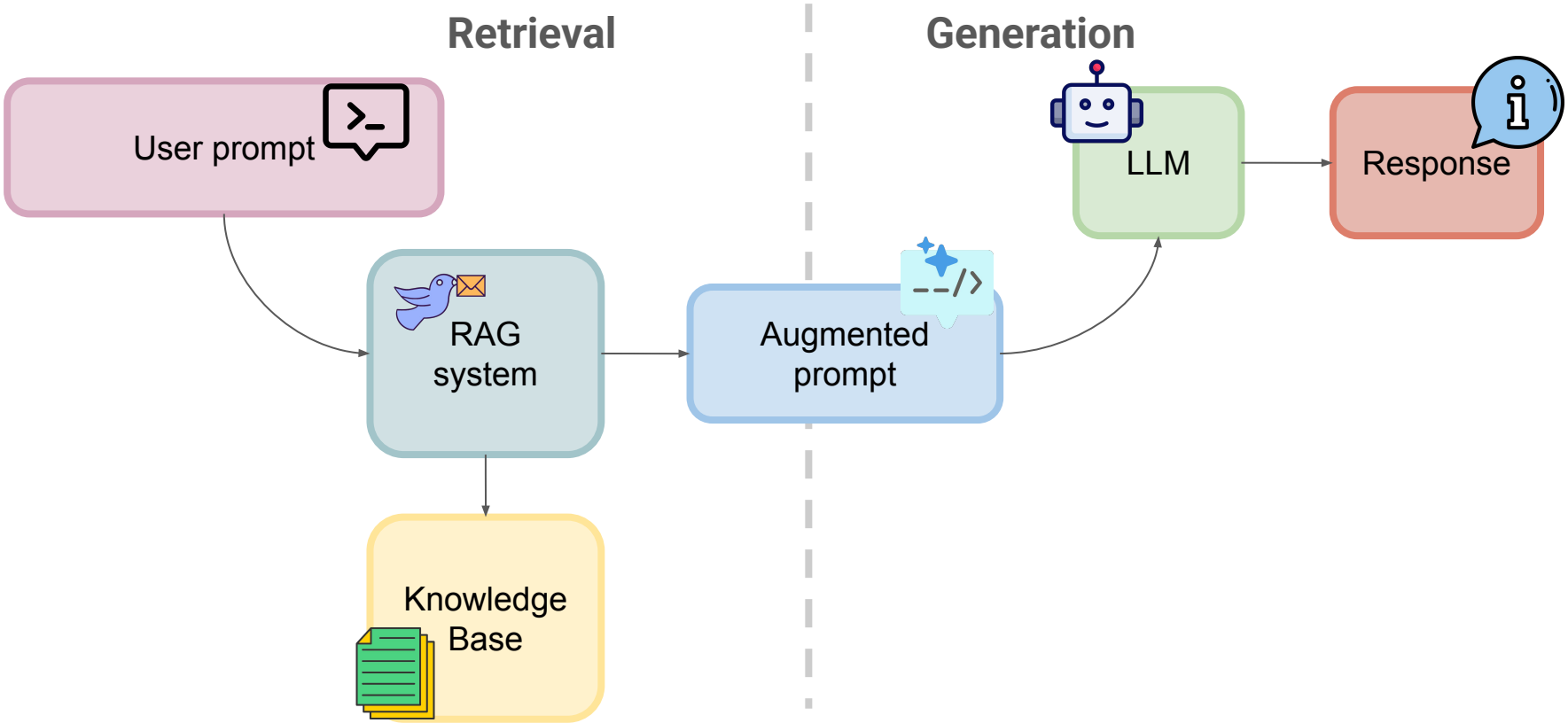
Retrieval Augmented Generation (RAG)
Module 2 Lecture 4

Pawan Goyal
Department of Computer Science and Engineering
IIT Kharagpur

RAG architecture - *Recap From the last lecture*



RAG Workflow

Advanced RAG: On top of the naive RAG workflow, we can add a pre-retrieval and a post retrieval step

Pre-retrieval: rewrites user query

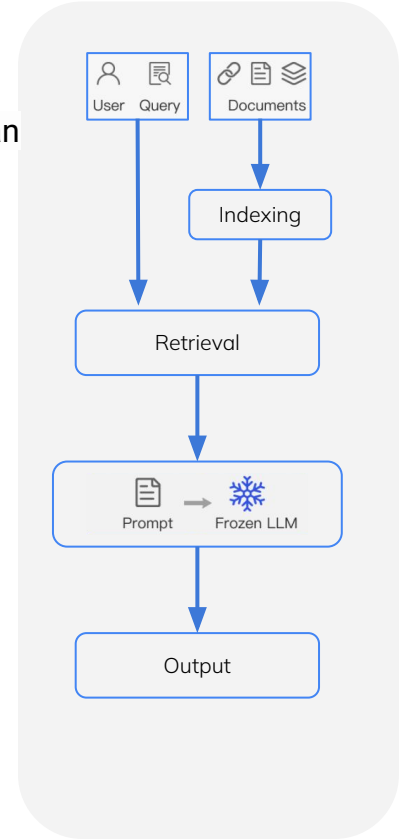
Benefits:

- ➢ Match domain-specific terminologies
- ➢ Provide additional context

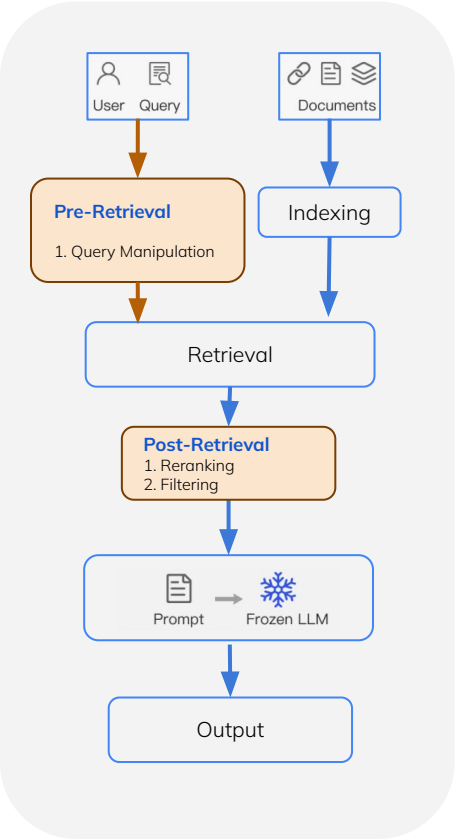
Post-retrieval: improves quality of added documents

Benefits:

- ➢ Reranks retrieved documents to reject irrelevant ones
- ➢ Filter out low-value contents



Naive RAG



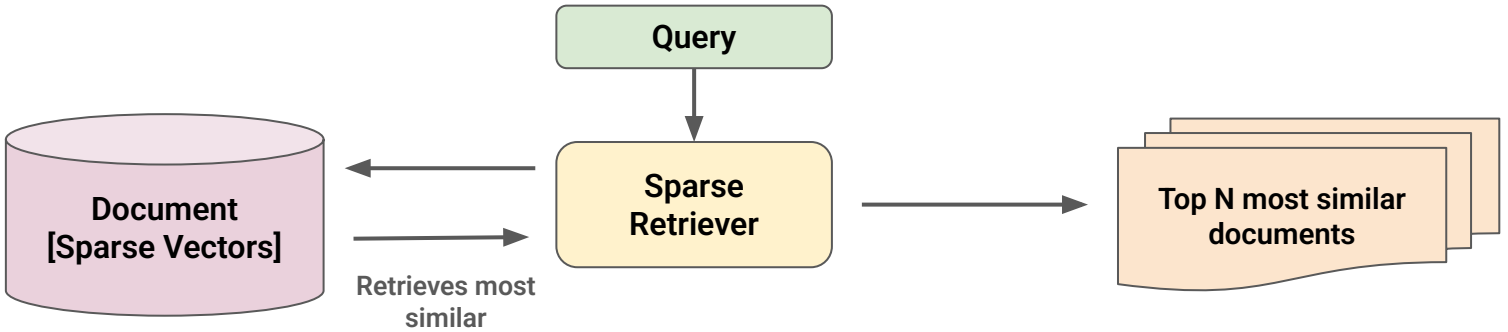
Advanced RAG

Sparse Retrieval

Sparse Retrieval in RAG: Keyword-Focused Search

Method: Matches documents based on shared keywords with the user's query.

Ranking: Uses algorithms like **tf-idf** and **BM25** to score documents based on how often and how uniquely query terms appear.



Strengths: Fast and efficient, especially for large datasets.

➢ Sparse retrieval methods are often more interpretable, as they provide clear reasoning based on term occurrences, making it easier to understand why certain documents were retrieved.

Limitations of Sparse Representations - *Recap from Module 1*

🔤 No Semantic Meaning

"king" and "queen" are just two unrelated dimensions

```
"cat" ≠ "kitten" in TF-IDF
```

📏 Curse of Dimensionality

Vocabulary of 100k words = 100k-dim sparse vectors

```
99%+ entries are zero
```

🔀 No Word Order

"dog bites man" and "man bites dog" are the same

```
Bag-of-words assumption
```

🆕 Out-of-Vocabulary

New words at test time create unknown slots

```
Zero vector = no info
```

→ *These are fundamental flaws. We need a smarter representation.*

Why Sparsity Breaks Similarity

The dot product between any two one-hot vectors is always zero

Cosine Similarity between one-hot vectors:



💡 **Insight**

One-hot vectors are orthogonal by construction — the model can never discover that "cat" and "kitten" are related.
We need a representation where similar words are geometrically close.

Problem with Sparse Retrieval

Sparse retrieval matches **strings**. Dense retrieval matches **meaning**.

QUERY

"How much vacation do I get?"

DOCUMENT

"Employees accrue 18 days of paid leave annually."

BM25 / TF–IDF: zero shared words → score ≈ 0. The right answer is invisible.

Dense embeddings: "vacation" and "paid leave" sit close in vector space → high similarity.

This vocabulary-mismatch gap is the whole reason dense retrieval exists.

Dense Representations

Replace one sparse index with a short, dense vector where every dimension carries meaning

SPARSE (One-Hot)

0	0	1	0	0	0	0	0	0	0	0	0

… (50,000 dims)

✗ 99%+ zeros

✗ No similarity

✗ Huge memory

learn weights →

DENSE (Word Embedding)

0.32	-0.71	0.15	0.88	-0.43	0.07	0.61

(300 dims — every value carries meaning)

✓ All values active

✓ Geometry = meaning

✓ Compact & fast

Geometry encodes meaning — famous examples:

```
vec("king") - vec("man") + vec("woman")   ≈ vec("queen")
vec("Paris") - vec("France") + vec("Germany")   ≈ vec("Berlin")
cosine( vec("cat"), vec("kitten") )   ≈ 0.92  ✓
```

Vector Embeddings

Words

word2vec · GloVe

“bank” — river or money?

static: one vector per word, context-blind



Sentences

SBERT · E5 · GTE

“How do I open an account?”

contrastively trained: paraphrases pulled together



Chunks

long-context embedders · late chunking

a paragraph of your document

the unit of retrieval in RAG — this is what we index



Anything

CLIP · CLAP · CodeBERT

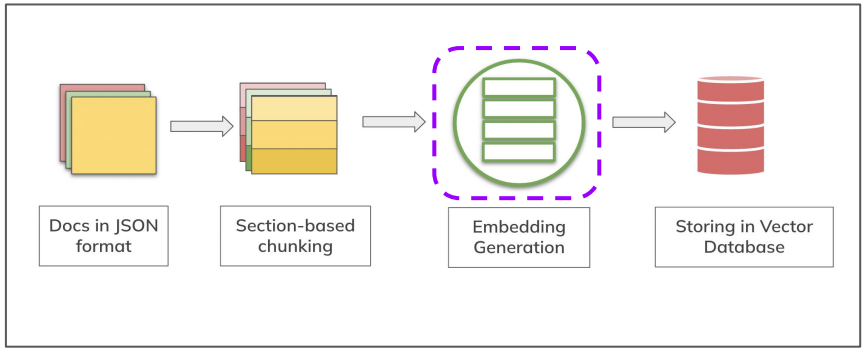
images · audio · code · tables

one shared space: a photo of a dog ≈ the word “dog”

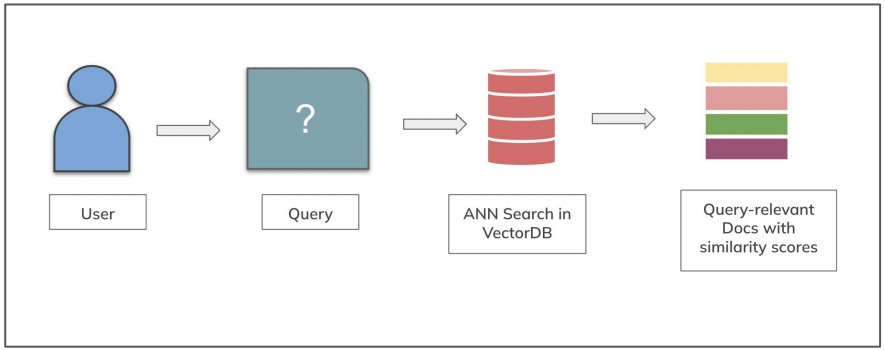


RAG with Embeddings: Pipeline

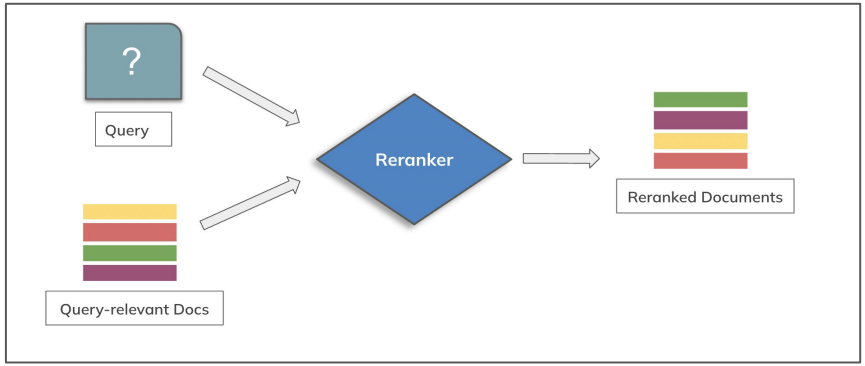
Pre-Retrieval



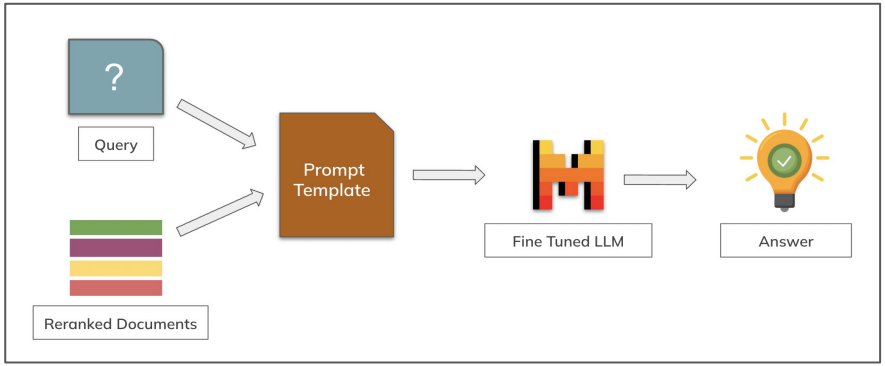
Retrieval



Post-Retrieval

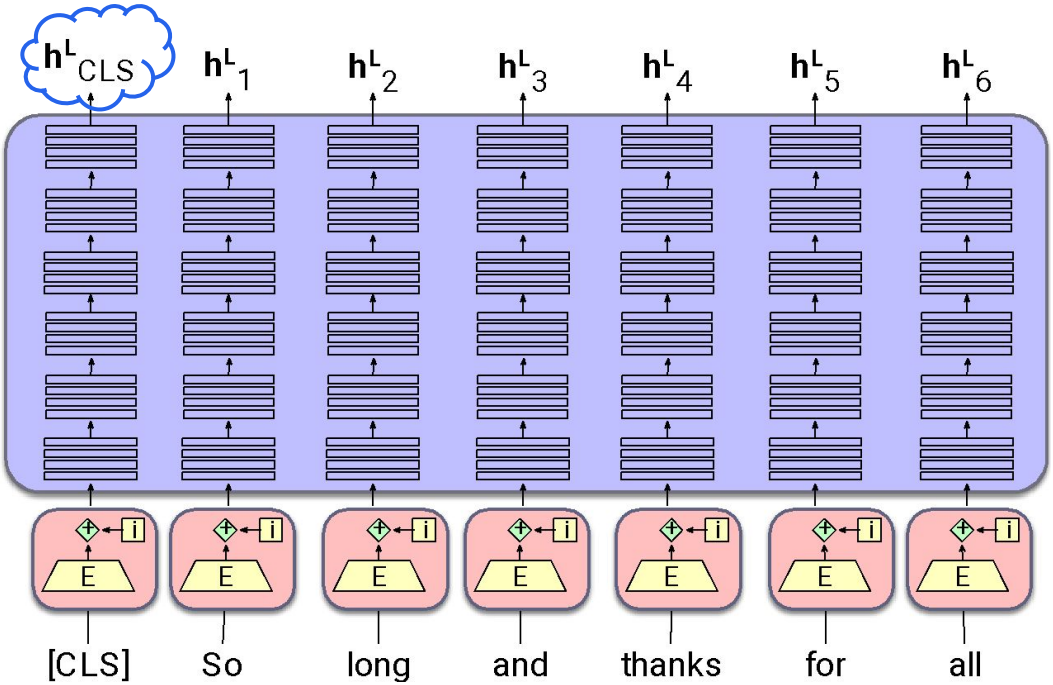


Augmentation and Generation



https://ai.meta.com/blog/retrieval-augmented-generation-streamlining-the-creation-of-intelligent-natural-language-processing-models/

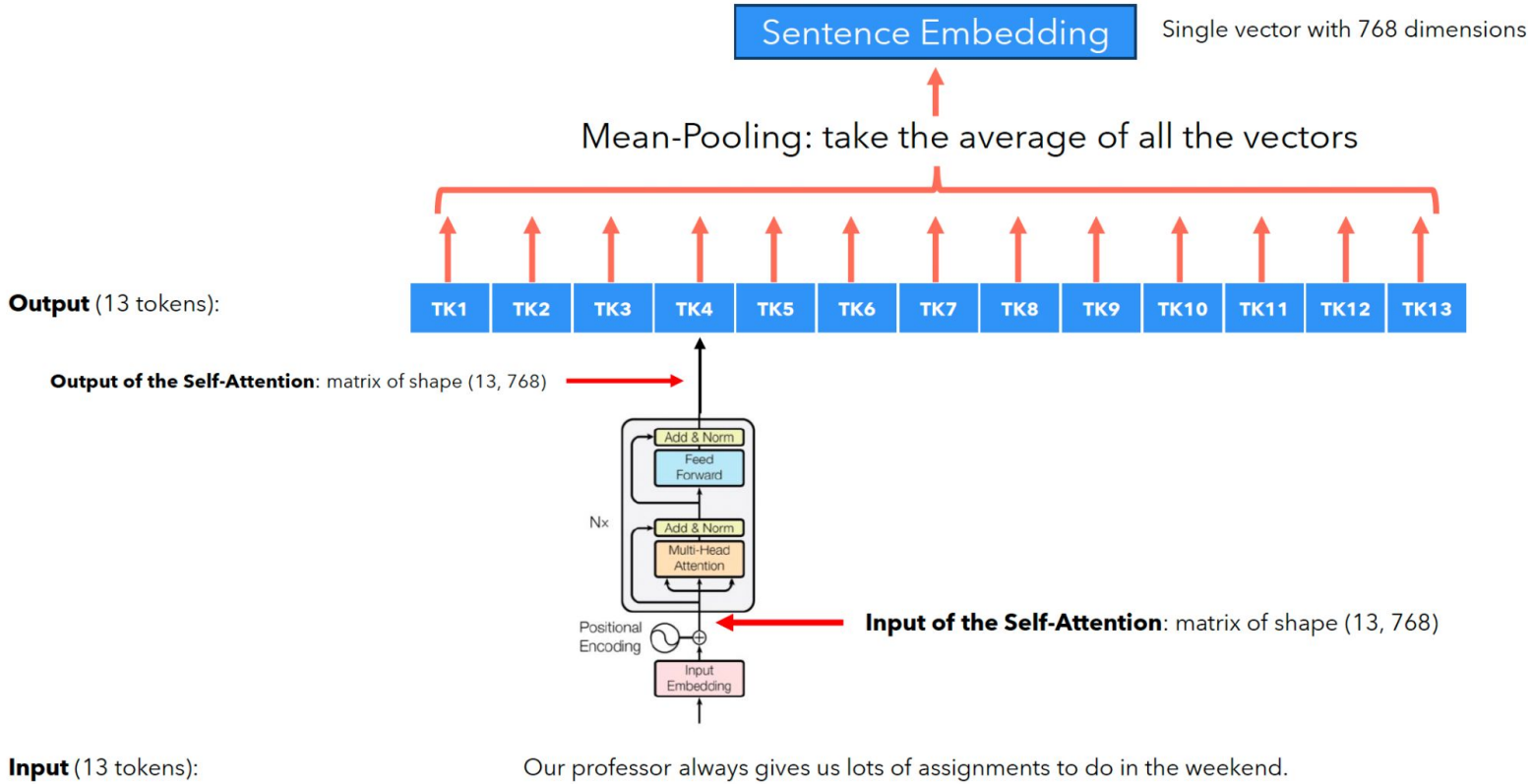
How do we get sentence embeddings using BERT?



We can get sentence embeddings from BERT using the [CLS] token, which is basically the whole sentence representation

https://web.stanford.edu/~jurafsky/slp3/

Sentence Embedding with BERT



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

Sentence Embeddings: Comparison

Comparing Sentence Embeddings

- Sentence embeddings can be compared using **cosine similarity**.
- Cosine similarity measures the angle between two embedding vectors.
- Smaller angles correspond to higher similarity scores.

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Challenge with BERT

- BERT is **not explicitly trained** to produce embeddings that are directly comparable using cosine similarity.
- Similar sentences are not guaranteed to have embeddings pointing in the same direction.
- This motivates methods for learning retrieval oriented embeddings such as **Sentence-BERT (SBERT)**, which train embeddings specifically for semantic similarity tasks.

Learning Retrieval-Oriented Embeddings

- Construct **positive** (relevant) and **negative** (irrelevant) document pairs.
- Train the encoder using a **contrastive / hinge loss** objective.
- Encourage higher similarity between the query and relevant documents than between the query and irrelevant documents.

$$\mathcal{L}(\theta, q) = \sum_{d_{pos} \in D_{pos}} \sum_{d_{neg} \in D_{neg}} \max\left(0,\ s(q, d_{neg}; \theta) - s(q, d_{pos}; \theta)\right)$$

Intuition

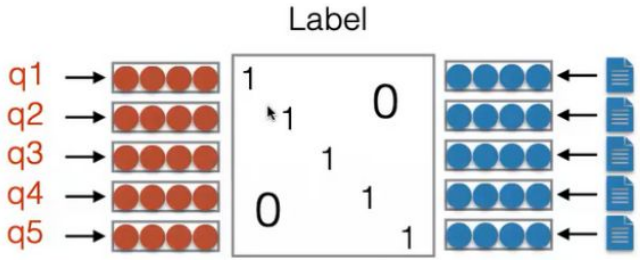
- Pull **relevant documents** closer to the query in embedding space.
- Push **irrelevant documents** farther away.
- Learn embeddings optimized for **retrieval and ranking tasks**.

How to Get Negative Examples?

In-batch negatives provide an efficient way to generate negative samples during training without requiring explicit negative mining.

Advantage:

- Scales efficiently without manually collecting negative examples.



Limitation:

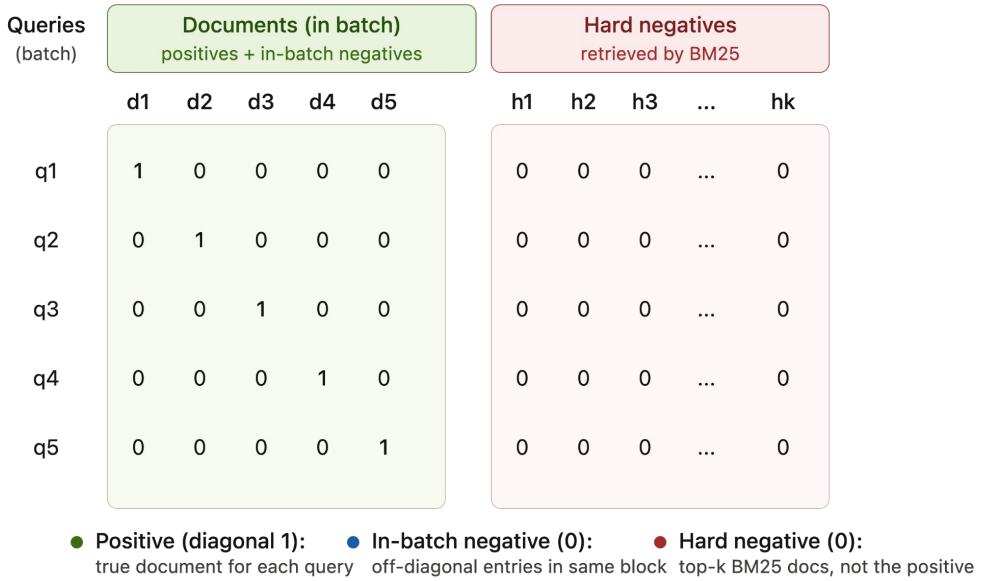
- Most in-batch negatives are often too easy, providing limited training signal.
- Additional techniques such as **hard negative mining** are often needed to improve retrieval performance.

Hard Mining

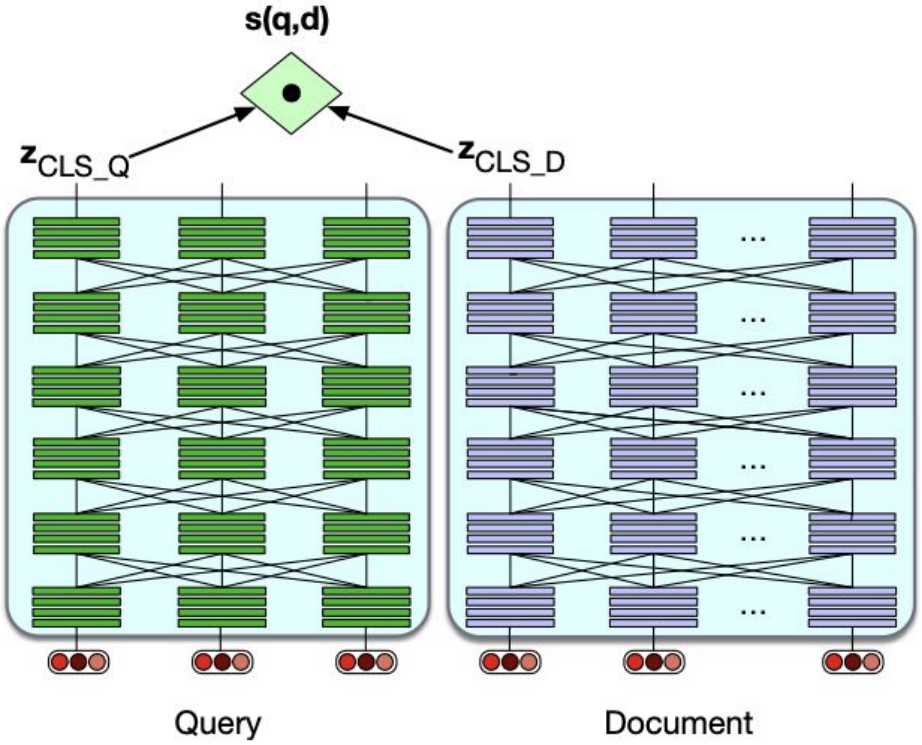
Hard negatives are documents that appear relevant to a query but are actually incorrect. They provide a stronger learning signal than random negatives.

Types of Negatives:

- **In-batch negatives:** Other documents within the training batch.
- **Hard negatives:** Using a weak retriever (BM25).

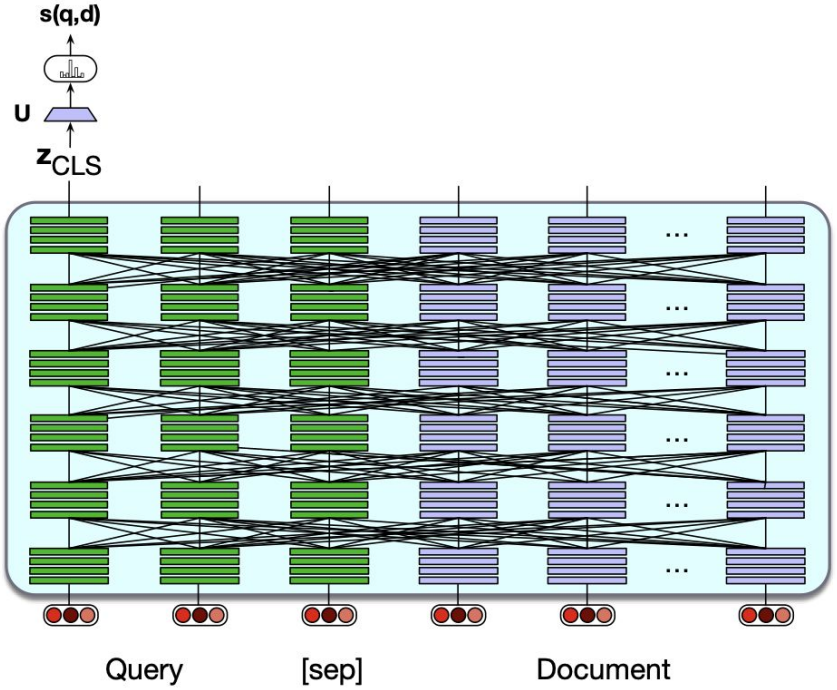


Dense Retrieval: Bi-Encoder



Use separate encoders for query and document, and use the dot product between CLS token outputs for the query and document as the score. This is less compute-expensive, but not as accurate.

Dense Retrieval (costlier but more accurate): Single Encoder

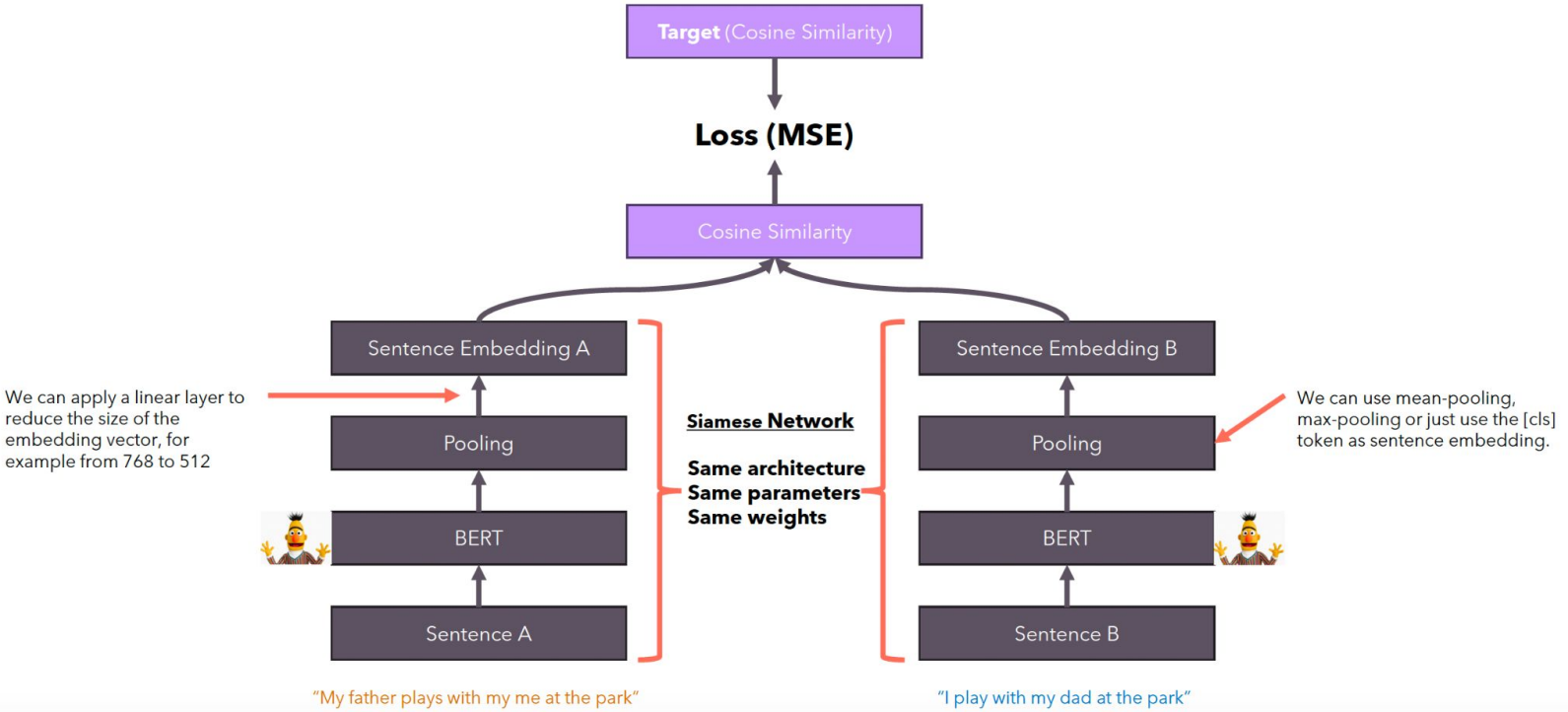


$$\mathbf{z} = \text{BERT}(\text{q}; [\text{SEP}]; \text{d})[\text{CLS}]$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

Use a single encoder to jointly encode query and document and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring

Sentence BERT: Architecture



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

Common Embedding Models

Model	Size	Architecture
BAAI/bge-base-en-v1.5	110M	BERT
intfloat/e5-base-v2	110M	RoBERTa
nomic-ai/nomic-embed-text-v1	~500M	GPT-style
sentence-transformers/all-MiniLM-L6-v2	22M	MiniLM

For Prototyping : all-MiniLM-L6-v2

MiniLM v2: "MiniLMv2: Multi-Head Self-Attention Relation Distillation for Compressing Pretrained Transformers". The paper generalizes deep self-attention distillation in MiniLMv1 by using self-attention relation distillation for task-agnostic compression of pre-trained Transformers.

[Multilingual] Pre-trained Models

Model	Teacher Model	Speedup	#Param	XNLI (Acc)	MLQA (F1)
L12xH384 mMiniLMv2	XLMR-Large	2.7x	117M	72.9	64.9
L6xH384 mMiniLMv2	XLMR-Large	5.3x	107M	69.3	59.0

OpenAI text-embedding-3-small

text-embedding-3-small is OpenAI's small text embedding model, creating embeddings with 1536 dimensions.

Matryoshka Representation: shorten vectors via the dimensions param with graceful accuracy loss.

```python
from openai import OpenAI

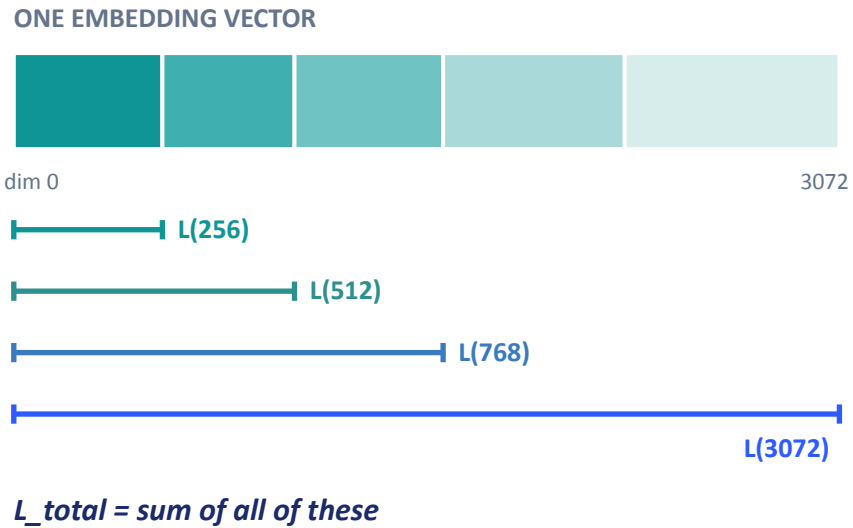
client = OpenAI()

response = client.embeddings.create(
    input="Your text string goes here",
    model="text-embedding-3-small"
)

print(response.data[0].embedding)
```

How Matryoshka embeddings order their dimensions

Same model, same forward pass. The trick is the **loss**: grade nested prefixes of the vector all at once.



Why early dims win

Dims 0–255 are scored by ***every*** term; the last dims by only one. Early dimensions feel more pressure, so the model packs the most broadly useful information there.

The analogy

Describe an animal, but you're cut off after an unknown number of words. You **front-load**: "small furry mammal…" first, fine details last. MRL hides where the cutoff is — so the model leads with what matters most.

Takeaway: the coarse-to-fine ordering is ***emergent*** - a side effect of grading every prefix. A non-MRL model never felt this pressure, so slicing its first 512 dims shreds the embedding.

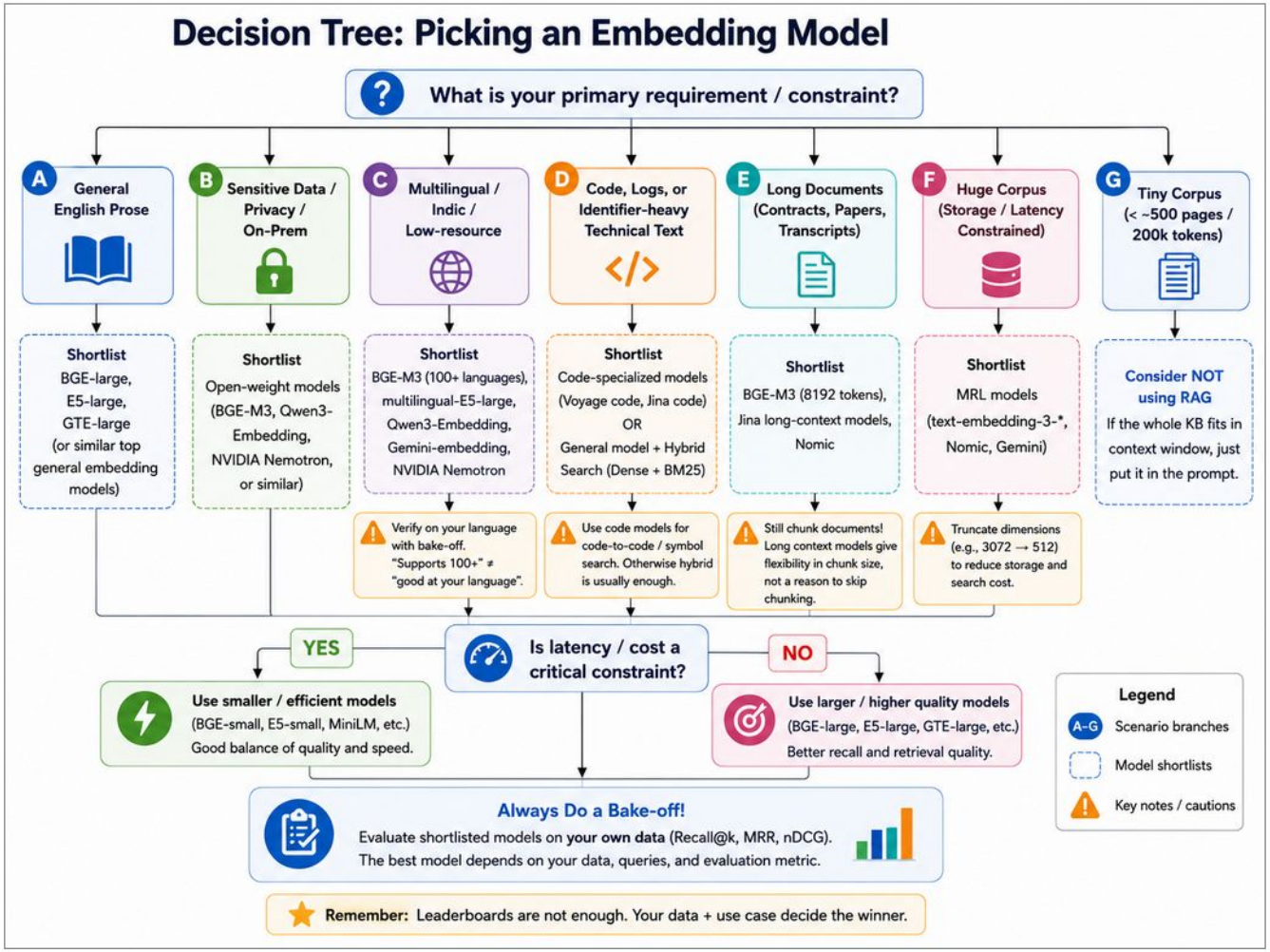
Choosing an Embedding Model

	SBERT (open-source)	OpenAI text-embedding-3
Hosting	Self-hosted / on-device	Hosted API only
Data privacy	Stays on your infra	Text sent to provider
Cost model	Compute you own (free lib)	Pay per token
Typical dims	384 – 768 (MiniLM, MPNet)	up to 3072 (resizable)
Customisation	Fine-tune on your domain	Fixed; prompt-only
Best when	Privacy, scale, custom domains	Fast start, top general quality

Design Decisions: Picking an Embedding Model

1. **The answer:** don't pick by the leaderboard - pick by **measuring candidates on your own data**. MTEB rankings are volatile and a high average score hides which model is strong on *your* task and *your* domain.

2. **The decision structure:** the shortlist itself is driven by properties of your dataset and your deployment constraints.


Decision Tree: Picking an Embedding Model
What is your primary requirement / constraint?
A General English Prose
Shortlist
BGE-large, E5-large, GTE-large (or similar top general embedding models)
B Sensitive Data / Privacy / On-Prem
Shortlist
Open-weight models (BGE-M3, Qwen3-Embedding, NVIDIA Nemotron, or similar)
C Multilingual / Indic / Low-resource
Shortlist
BGE-M3 (100+ languages), multilingual-E5-large, Qwen3-Embedding, Gemini-embedding, NVIDIA Nemotron
Verify on your language with bake-off. "Supports 100+" ≠ "good at your language".
D Code, Logs, or Identifier-heavy Technical Text
Shortlist
Code-specialized models (Voyage code, Jina code) OR General model + Hybrid Search (Dense + BM25)
Use code models for code-to-code / symbol search. Otherwise hybrid is usually enough.
E Long Documents (Contracts, Papers, Transcripts)
Shortlist
BGE-M3 (8192 tokens), Jina long-context models, Nomic
Still chunk documents! Long context models give flexibility in chunk size, not a reason to skip chunking.
F Huge Corpus (Storage / Latency Constrained)
Shortlist
MRL models (text-embedding-3-*, Nomic, Gemini)
Truncate dimensions (e.g., 3072 → 512) to reduce storage and search cost.
G Tiny Corpus (< ~500 pages / 200k tokens)
Consider NOT using RAG
If the whole KB fits in context window, just put it in the prompt.
Is latency / cost a critical constraint?
YES
Use smaller / efficient models
(BGE-small, E5-small, MiniLM, etc.)
Good balance of quality and speed.
NO
Use larger / higher quality models
(BGE-large, E5-large, GTE-large, etc.)
Better recall and retrieval quality.
Always Do a Bake-off!
Evaluate shortlisted models on your own data (Recall@k, MRR, nDCG).
The best model depends on your data, queries, and evaluation metric.
Remember: Leaderboards are not enough. Your data + use case decide the winner.
Legend
A–G Scenario branches
Model shortlists
Key notes / cautions

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch A - General English prose, you want zero infrastructure

Shortlist: hosted APIs - OpenAI `text-embedding-3-small/large`, Voyage `voyage-3-large`, Cohere `embed-v4`, Google `gemini-embedding`.

Why: you send text, you get vectors : no GPU, no model hosting, no version pinning. For most business RAG this is the right *first* move; the per-token cost is trivial next to an engineer's time. `text-embedding-3-small` (1536-dim) is the cheap safe default; go `-large` (3072-dim) only if the bake-off shows it earns the cost.

Branch B - Data is sensitive / must stay on-prem (privacy, compliance)

Shortlist: open-weight local - `BGE-M3`, `Qwen3-Embedding` (0.6B / 4B / 8B), `NV-Embed-v2`, `multilingual-e5-large`.

Why: healthcare, legal, finance, or internal corporate data often *legally cannot* be sent to a third-party API. Open weights run inside your VPC; the only cost is the GPU you already have. Open-source quality has largely closed the gap with commercial APIs on retrieval. **The size dial:** Qwen3-Embedding ships at 0.6B / 4B / 8B. Bigger = better recall but more VRAM, slower, costlier per embed. Start at the smallest that passes your eval - an 8B model that beats a 0.6B by 1% recall but is 5× slower is usually a bad trade for RAG.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch C - Multilingual / Indic / low-resource languages

Shortlist: `BGE-M3` (100+ languages), `multilingual-e5-large`, `Qwen3-Embedding`, NVIDIA `Llama-Embed-Nemotron-8B`, `gemini-embedding`.

Why: a model trained mostly on English will map two sentences that mean the same thing in Hindi and English to *distant* vectors - cross-lingual retrieval silently fails. Multilingual models are trained so meaning, not language, drives proximity. **The non-negotiable caveat:** "supports 100+ languages" ≠ "good at *your* language." Coverage for high-resource languages (Hindi, Arabic) is far better than for low-resource ones (e.g., Sanskrit, many African languages).

You must verify on your language with the bake-off - this is the branch where the leaderboard lies most. If retrieval is weak, fine-tuning a multilingual base on in-domain pairs often beats swapping models.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch D - Code, logs, or identifier-heavy technical text

Shortlist: code-specialized embedders (e.g. Voyage code / Jina code models) **or** a general model **plus hybrid search**.

Why: code and logs are full of exact tokens — function names, error codes, SKUs that dense semantic embeddings handle poorly . Two fixes: a model pretrained on code, *or* lean on BM25 in a hybrid setup to catch the exact tokens. Often the cheapest win is **general embedder + hybrid**, not a specialist model.
Decision rule: if your queries are mostly natural-language questions *about* code, a general model + hybrid is usually enough; if you're doing code-to-code or symbol search, a code-specialized model earns its place.

Branch E - Long documents (legal contracts, papers, transcripts)

Check first: the model's `max_seq_length`. Many embedders cap at 512 tokens and **silently truncate** longer input — you embed only the first paragraph and lose the rest.
Shortlist for long context: `BGE-M3` (8192 tokens), Jina long-context models, Nomic.
Why it's usually a chunking decision anyway: even with an 8k-token model, embedding a whole contract into one vector dilutes it. Long-context models mostly buy you *flexibility in chunk size*, not the right to skip chunking. The real choice is "chunk small + standard model" vs "chunk large + long-context model," and the bake-off decides.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch F - Huge corpus, storage/latency constrained

Shortlist: models with **Matryoshka representation learning (MRL)** - `text-embedding-3-*`, Nomic, Gemini - so you can *truncate* dimensions.

Why: storage and search cost scale with `dimensions × vectors`. At 100M vectors, 3072-dim float32 is ~1.2 TB; truncating to 512-dim cuts that ~6× for a small recall hit. MRL models are trained so the first *k* dimensions are already a usable embedding - you just slice the vector.

```python
# Matryoshka truncation: take the first N dims, then RE-NORMALIZE (critical).
import numpy as np
def truncate(vecs, dim):
    v = vecs[:, :dim]
    return v / np.linalg.norm(v, axis=1, keepdims=True)  # WHY renorm: cosine needs unit length
# Sweep dim in the bake-off: find the smallest dim that still passes your recall bar.

```

The tradeoff: dimensions are a direct storage/quality dial. Don't pay for 3072 if 768 passes your eval - but don't truncate a model that *wasn't* trained with MRL, or you'll shred the embedding.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch F - Huge corpus, storage/latency constrained

Shortlist: models with **Matryoshka representation learning (MRL)** - `text-embedding-3-*`, Nomic, Gemini — so you can *truncate* dimensions.

Why: storage and search cost scale with `dimensions × vectors`. At 100M vectors, 3072-dim float32 is ~1.2 TB; truncating to 512-dim cuts that ~6× for a small recall hit. MRL models are trained so the first *k* dimensions are already a usable embedding - you just slice the vector.

Branch G - Tiny corpus (< ~500 pages / 200k tokens)

Consider not doing RAG at all. If the whole knowledge base fits in the model's context window, just put it in the prompt - no embedder, no index, no retrieval failures. RAG is the solution for corpora too big for context; below that threshold it's complexity you don't need.

The #1 silent production RAG bug

Recall is mediocre, so the team swaps to a fancier embedding model… and it barely helps.

The real cause

A missing query: / passage: prefix on an asymmetric model (E5, BGE, GTE, Qwen, Instructor, Jina). The model treats a question like a document and collapses the asymmetry it was trained to use.

The fix (free)

Read the model card. Add the exact required prefixes. Recovers more recall than most model swaps would have gained — at zero cost.

Important point: *before reaching for a better model, verify you're using the current one correctly. The cheapest wins are configuration, not architecture.*

```python
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("intfloat/multilingual-e5-large")

# WRONG — same encoding for both. Recall drops, no warning.
# q_emb = model.encode(query); d_emb = model.encode(documents)

# RIGHT — E5/BGE-style models need these exact prefixes:
q_emb = model.encode([f"query: {query}"],     normalize_embeddings=True)
d_emb = model.encode([f"passage: {d}" for d in documents], normalize_embeddings=True)
```

Two Decisions That Make or Break RAG Search

Match your search to how your model was trained—and to your data.

1 DISTANCE METRIC – LET THE MODEL DECIDE

The metric isn't a free choice – it must match how the model was trained. Cosine/inner-product for the normalized models above; Euclidean is rarely right for text and quietly underperforms.

The bug is using Euclidean on a cosine-trained model "because the DB defaulted to it."

MATCH THE METRIC TO THE MODEL

MODEL / EMBEDDING TYPE	USE THIS METRIC
Normalized embeddings (OpenAI, Cohere, many SBERT variants, etc.)	Cosine Similarity (or Inner Product) ✓
Unnormalized embeddings (Some models, custom, or legacy)	Euclidean Distance (rarely best for text) ⚠

Rule of thumb: If your embeddings are normalized to unit length, use Cosine (or Inner Product). Only reach for Euclidean with a strong reason (e.g., non-text data or specific model guidance).

2 DENSE-ONLY vs HYBRID – DRIVEN BY IDENTIFIER DENSITY

The decision variable is how much of your corpus's value lives in exact tokens dense search can't feel.

LOW IDENTIFIER DENSITY
(Mostly prose, concepts, natural language)

HIGH IDENTIFIER DENSITY
(Codes, IDs, names, jargon, acronyms, exact terms)

DENSE-ONLY IS OFTEN FINE ✓

- Natural-language FAQ
- Explanatory content, how-tos, guides
- Conceptual understanding is the primary need

HYBRID (DENSE + BM25) IS BETTER ✓

- Product codes, part numbers, order IDs
- Names, acronyms, legal terms, medical or technical jargon
- Anything where exact token matches carry meaning

Don't add BM25 complexity to a corpus that has no exact-match queries; don't omit it from one that does.

THE TAKEAWAY

- Use the right metric for how your model was trained. Cosine/inner-product for normalized text embeddings. Euclidean is rarely the answer.
- Choose dense-only or hybrid based on identifier density. Match the search to your data, not the other way around.

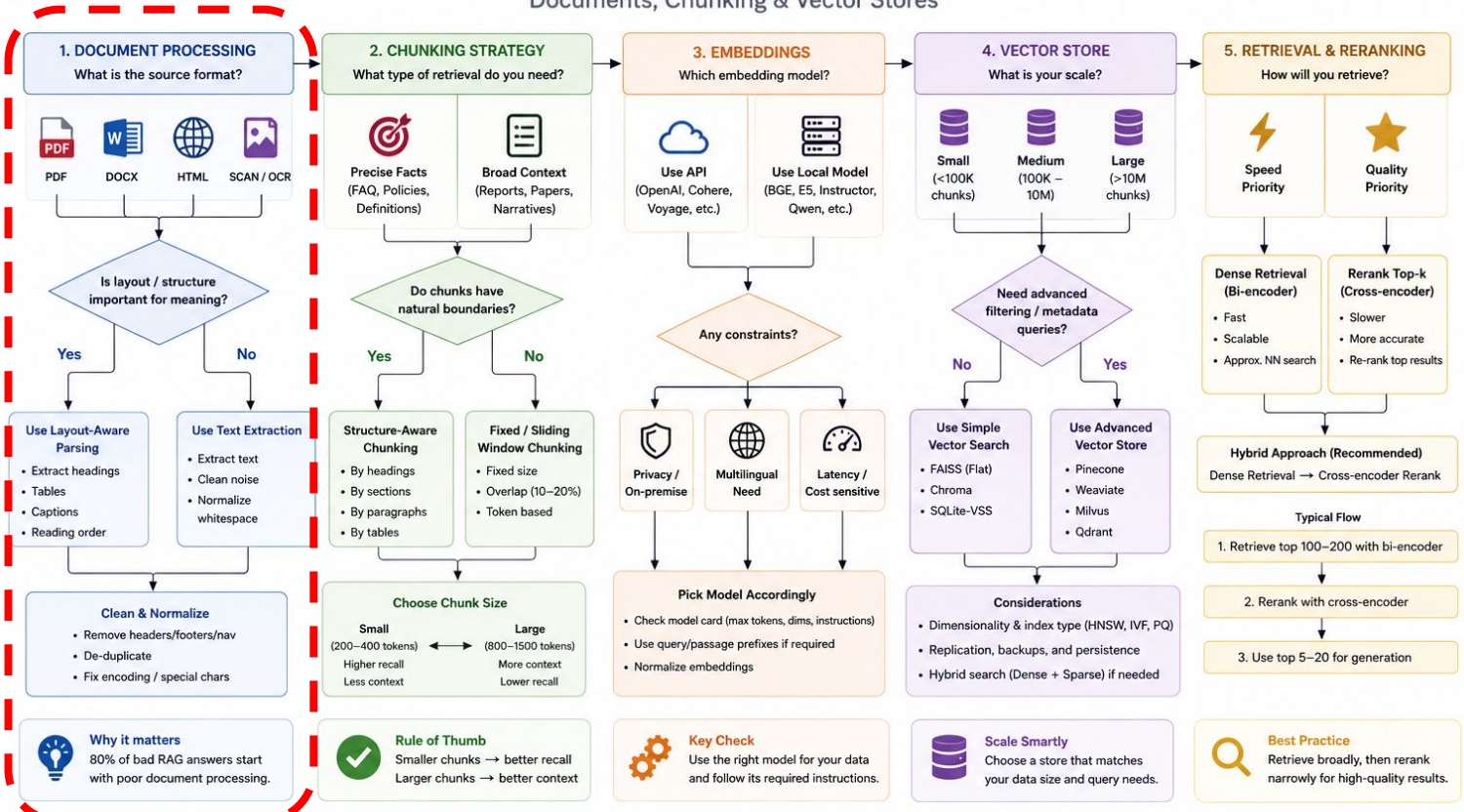
The Unifying Principle

Every embedding choice is decided by a condition.

	Choice	The condition that decides it
	**Hosted vs local embedder**	 Can the **data leave** your network? Corpus size vs API cost.
	**Which multilingual model**	 Your specific language's coverage — **measured**, not advertised.
	**Code-specialized vs general+hybrid**	 Are queries about **code**, or code-to-code?
	**Long-context model vs small chunks**	 `max_seq_length` vs your chunk strategy.
	**Embedding dimensions (MRL)**	 **Storage/latency budget** at your vector count.
	**Query/passage prefixes**	 Is the model **symmetric** or **asymmetric**? (Read the card.)
	**Distance metric**	 What the model was **trained for**.
	**Dense vs hybrid**	 Identify **density** in the corpus.
	**RAG vs just prompt**	 Does the corpus **fit in context**?

Design Choices in RAG Systems

Documents, Chunking & Vector Stores



Key Principle Good RAG is about making the right design choices at every step — clean data, meaningful chunks, appropriate embeddings, scalable storage, and effective retrieval.

Document Processing Basics

Garbage in, garbage out. *Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR

Document Processing Basics

***Garbage in, garbage out.** Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR



Structured XML under the hood
Headings, lists, tables already labelled
Easiest, most reliable to parse

Document Processing Basics

***Garbage in, garbage out.** Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR



Structured XML under the hood
Headings, lists, tables already labelled
Easiest, most reliable to parse



Rich semantic tags (h1, p, table)
But full of boilerplate
Strip nav / headers / footers / sidebars

Document Processing Basics

***Garbage in, garbage out.** Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR



Structured XML under the hood
Headings, lists, tables already labelled
Easiest, most reliable to parse



Rich semantic tags (h1, p, table)
But full of boilerplate
Strip nav / headers / footers / sidebars

Extraction pulls raw characters. Parsing goes further labelling what each block is (heading, table row, caption, footnote) and keeping the relationships. For RAG, that structure is as valuable as the text.

Document Processing Basics : OCR

OCR (optical character recognition) recovers text from scanned/image PDFs.

Many real-world PDFs are essentially pictures of text without OCR there's nothing to embed.

Tables are the hardest element to extract faithfully.

Best practice: emit tables as HTML to preserve row/column structure (often better than markdown).

Layout-aware models recover reading order, columns, and hierarchy, not just a flat text dump.

Layout-aware toolkits

- **Unstructured** — Element ontology + Hi-Res layout; PDF/DOCX/HTML/images
- **Docling (IBM)** — DocLayNet layout + TableFormer tables; OCR built in
- **LlamaParse** — LLM-based parsing; lightweight, good on simpler docs
- **Cloud OCR** — Azure Document Intelligence, Google Document AI

Design Decisions: Documents

The governing question: *what physical form is the text in, and how much of its value is in layout?* Get this wrong and every downstream step inherits the damage - 80% of bad RAG answers are decided here.

Design Decisions: Documents

1. **PDF, plain prose (papers, reports, specs)**

Choose: `pypdf`. **Because:** it's 5–10× faster and lighter than anything else, and plain prose has no structure to preserve. Don't reach for a heavier tool to solve a problem you don't have.

2. **PDF whose value is in TABLES (invoices, financials, datasheets)**

Choose: `pdfplumber` (or Camelot for ruled tables), serialize tables to Markdown. **Because:** `pypdf` flattens a table into a single word-salad line (`Q1 100 Q2 120 …`), destroying the row/column meaning your retriever needs. The speed cost buys back the numbers you came for.

Design Decisions: Documents

3. Scanned / image-only PDF (no text layer)

Choose: OCR - local `tesseract` (+ `pdf2image`), or cloud OCR / a vision model for hard scans.
Because: there is literally no text to extract; `pypdf` returns empty strings *silently*.
Always probe first - this is the trap that ships empty chunks to production:

```
def pdf_has_text_layer(path, sample=3):
        import pypdf
        r = pypdf.PdfReader(path)
        return any((p.extract_text() or "").strip() for p in r.pages[:sample])
# False -> route to OCR, never to pypdf.
```

The DPI / preprocessing why: OCR at `dpi=300` is the sweet spot (Tesseract was tuned near it); 72 dpi shreds small fonts, 600 dpi is ~4× slower for little gain. And cleaning the image (grayscale + binarization) usually helps accuracy *more* than adding pixels. The cheap win is a cleaner image, not a bigger one.

Design Decisions: Documents

4. Complex layout (multi-column, magazines, academic PDFs, forms)

Choose: a layout-aware parser - `Unstructured`, `docling`, `GROBID` (for papers), Azure Document Intelligence, or a VLM for the truly messy ones. **Because:** `pypdf` reads coordinates left-to-right and **scrambles multi-column reading order** - column 1 line 1, column 2 line 1, column 1 line 2… producing interleaved nonsense.

5. HTML

- **You control the site / stable layout** → `BeautifulSoup` with targeted selectors. **Because:** high precision — you keep exactly what you target.
- **Scraping the open web at scale** → `trafilatura` / readability. **Because:** every site differs; you can't hand-write selectors for all of them, so you want a heuristic that generalizes (high recall across unknown layouts).

The shared why: raw HTML is ~90% navigation, footers, cookie banners, and ads. Strip boilerplate *before* extracting text or every chunk is polluted with "Home | About | Contact."

Design Decisions: Documents

Choosing the Right Tool for Document Parsing

Match your document type to the right tool for accurate, reliable extraction.

Document condition	Tool	Decided by (Why)
Plain text PDF	pypdf	Speed, no structure to keep
Table-heavy PDF	pdfplumber / Camelot	Tables die under pypdf
Scanned PDF	OCR (tesseract / VLM)	No text layer (probe first!)
Multi-column / forms	Unstructured / docling / Azure DI	Reading order + element tags
HTML, controlled	BeautifulSoup	Precision
HTML, open web	trafilatura	Robustness across layouts
DOCX	python-docx + headings	Free section metadata
Mixed at scale	router + Unstructured	One entry point

Key takeaway: Use the simplest tool that preserves the structure you need.
Garbage in → garbage out. Parsing quality directly impacts retrieval quality.

Design Choices in RAG Systems

Documents, Chunking & Vector Stores

1. DOCUMENT PROCESSING

What is the source format?

PDF · DOCX · HTML · SCAN / OCR

Is layout / structure important for meaning?

- **Yes** → **Use Layout-Aware Parsing**
  - Extract headings
  - Tables
  - Captions
  - Reading order
- **No** → **Use Text Extraction**
  - Extract text
  - Clean noise
  - Normalize whitespace

Clean & Normalize

- Remove headers/footers/nav
- De-duplicate
- Fix encoding / special chars

Why it matters

80% of bad RAG answers start with poor document processing.

2. CHUNKING STRATEGY

What type of retrieval do you need?

- **Precise Facts** (FAQ, Policies, Definitions)
- **Broad Context** (Reports, Papers, Narratives)

Do chunks have natural boundaries?

- **Yes** → **Structure-Aware Chunking**
  - By headings
  - By sections
  - By paragraphs
  - By tables
- **No** → **Fixed / Sliding Window Chunking**
  - Fixed size
  - Overlap (10–20%)
  - Token based

Choose Chunk Size

Small	⟷	Large
(200–400 tokens)		(800–1500 tokens)
Higher recall		More context
Less context		Lower recall

Rule of Thumb

Smaller chunks → better recall
Larger chunks → better context

3. EMBEDDINGS

Which embedding model?

- **Use API** (OpenAI, Cohere, Voyage, etc.)
- **Use Local Model** (BGE, E5, Instructor, Qwen, etc.)

Any constraints?

- **Privacy / On-premise**
- **Multilingual Need**
- **Latency / Cost sensitive**

Pick Model Accordingly

- Check model card (max tokens, dims, instructions)
- Use query/passage prefixes if required
- Normalize embeddings

Key Check

Use the right model for your data and follow its required instructions.

4. VECTOR STORE

What is your scale?

- **Small** (<100K chunks)
- **Medium** (100K – 10M)
- **Large** (>10M chunks)

Need advanced filtering / metadata queries?

- **No** → **Use Simple Vector Search**
  - FAISS (Flat)
  - Chroma
  - SQLite-VSS
- **Yes** → **Use Advanced Vector Store**
  - Pinecone
  - Weaviate
  - Milvus
  - Qdrant

Considerations

- Dimensionality & index type (HNSW, IVF, PQ)
- Replication, backups, and persistence
- Hybrid search (Dense + Sparse) if needed

Scale Smartly

Choose a store that matches your data size and query needs.

5. RETRIEVAL & RERANKING

How will you retrieve?

- **Speed Priority**
- **Quality Priority**

Dense Retrieval (Bi-encoder)

- Fast
- Scalable
- Approx. NN search

Rerank Top-k (Cross-encoder)

- Slower
- More accurate
- Re-rank top results

Hybrid Approach (Recommended)

Dense Retrieval → Cross-encoder Rerank

Typical Flow

1. Retrieve top 100–200 with bi-encoder
2. Rerank with cross-encoder
3. Use top 5–20 for generation

Best Practice

Retrieve broadly, then rerank narrowly for high-quality results.

Key Principle

Good RAG is about making the right design choices at every step — clean data, meaningful chunks, appropriate embeddings, scalable storage, and effective retrieval.

Why chunk? Right-sizing for retrieval

1. Documents are split into chunks, each embedded into one vector.
2. Similarity is computed chunk-by-chunk, so the chunk is the true unit of retrieval.
3. Embedding models also cap input length, forcing a split for long docs.

The goal

Each chunk should hold one self-contained, retrievable idea - small enough to be precise, large enough to stand alone.

Chunking (fixed-size · sliding window · semantic)

The governing question: *what is one self-contained "answer" in this corpus, and where are its natural seams?*

The eternal tension: too small loses context ("it rose 40%" - what rose?), too big dilutes the embedding and over-retrieves. Size should match how much text typically holds one complete answer in *your* data.

Chunking Strategies: Which One Should You Choose?

Three proven approaches to chunk your data for better retrieval.

	1. Fixed-size Simple. Fast. Predictable.	2. Sliding Window Balanced. Robust. Reliable.	3. Semantic Smart. Meaningful. Powerful.
Best For	Quick baseline, experimentation, uniform or structured content	Most RAG applications, general-purpose use	High-value knowledge bases, complex or topic-shifting content
How It Works	Split every N tokens / characters.	Fixed chunks that overlap (e.g., 512 tok, 50–100 overlap).	Split where the topic shifts (embedding distance between sentences).
Advantages	• Dead simple & fast • Predictable chunk count • Cuts mid-sentence / mid-idea • Good baseline to start from	• Carries context across boundaries • Nothing severed at the cut line • Redundant storage & duplicate hits • Pinecone's classic baseline	• Chunks align to meaning • Can lift recall ~9% vs. naive • ~14× slower — embeds every sentence • Pay the cost only when it helps
Limitations	Can break context across boundaries	More storage & duplicate retrievals	Higher compute & complexity to implement

How to Choose (Decision Guide)

1. Start Simple
Prototyping or just getting started?
→ **Use Fixed-size**
Fast, simple, predictable baseline.

2. Improve Quality
Need good retrieval quality for most use cases?
→ **Use Sliding Window**
Recommended default for most RAG apps.

3. Maximize Quality
Is retrieval quality business-critical and worth the extra cost?
→ **Use Semantic**
Highest quality when it matters most.

Rule of Thumb: Start with **Sliding Window Chunking**. Move to Semantic Chunking only when retrieval quality justifies the additional cost.

Source: Pinecone, "Chunking Strategies"; Stack Overflow, "Breaking up is hard to do"; Firecrawl, "Best Chunking Strategies for RAG (2026)."

1. Uniform prose, prototyping / a baseline

Choose: fixed-size, **token-based** (not character-based).
Because: simplest and fastest; you need a baseline to beat. Count tokens with *the embedding model's own tokenizer*, because the model's limit is in tokens - counting characters makes your budget a lie.

2. Prose where answers straddle cuts (most RAG)

Choose: sliding window with **~10–20% overlap**. **This is the sane default.**
Because: a fact spanning a chunk boundary gets split across two chunks and neither retrieves well; overlap guarantees it lands intact in at least one.

The dial: 0% = boundaries get cut; ~20% = catches them cheaply; 50%+ = near-duplicate chunks that double your store for marginal recall. 20% is the knee of the curve, not a magic number.

3. Strongly structured docs (Markdown, legal clauses, API docs, code)

Choose: structure-aware splitting - on headers, on clauses, on functions (AST for code).
Because: the document already tells you where the seams are. A chunk that *is* "§7.2 Termination" retrieves precisely for "termination clause" *and* carries provenance you can cite. A half-function or half-clause is useless to the LLM.

For code: split by function/class, never by token count
`RecursiveCharacterTextSplitter` has language-aware variants for this.

4. Heterogeneous, topic-shifting docs; quality > ingest cost

Choose: semantic chunking (cut where consecutive-sentence similarity drops).
Because: it puts boundaries at *meaning* shifts instead of arbitrary positions. The cost: you embed every sentence at ingest - expensive on huge corpora.
Always pair it with a hard `max_tokens` cap, or one long on-topic run blows the model limit.

Further Design Choice Decisions

1. Conversation, chat logs, transcripts (eg Customer helpline)

2. Dense reference text where the match needs surrounding context

3. Tables, CSVs, records

Tuning chunking - and an overlap myth

Chunk on natural boundaries - sentences & paragraphs beat arbitrary character limits.

Common starting point: ~512 tokens, 10–20% (50–100 tok) overlap.

Recursive splitting (LangChain) respects structure then falls back to size.

Match chunk size to content: small for Q&A facts, larger for narrative/code.

Always evaluate retrieval & iterate - there is no universal best size.



Test, don't assume



Overlap is treated as a universal default — but a Jan-2026 arXiv study (SPLADE + 8B Mistral on Natural Questions) found **overlap gave no measurable benefit in their setup, only higher indexing cost.**



Semantic chunking can be **~14× slower** than token splitting (Chonkie benchmark).

→ *Spend that budget only when your metrics actually move.*

Choosing the Right Chunker for Your Data

General RAG Prose

Source-Grounded: Text synthesized directly from documents in a retrieved knowledge base.Information Dense: Prioritizes facts, data, and citations over stylistic flair.

Uniform Prose

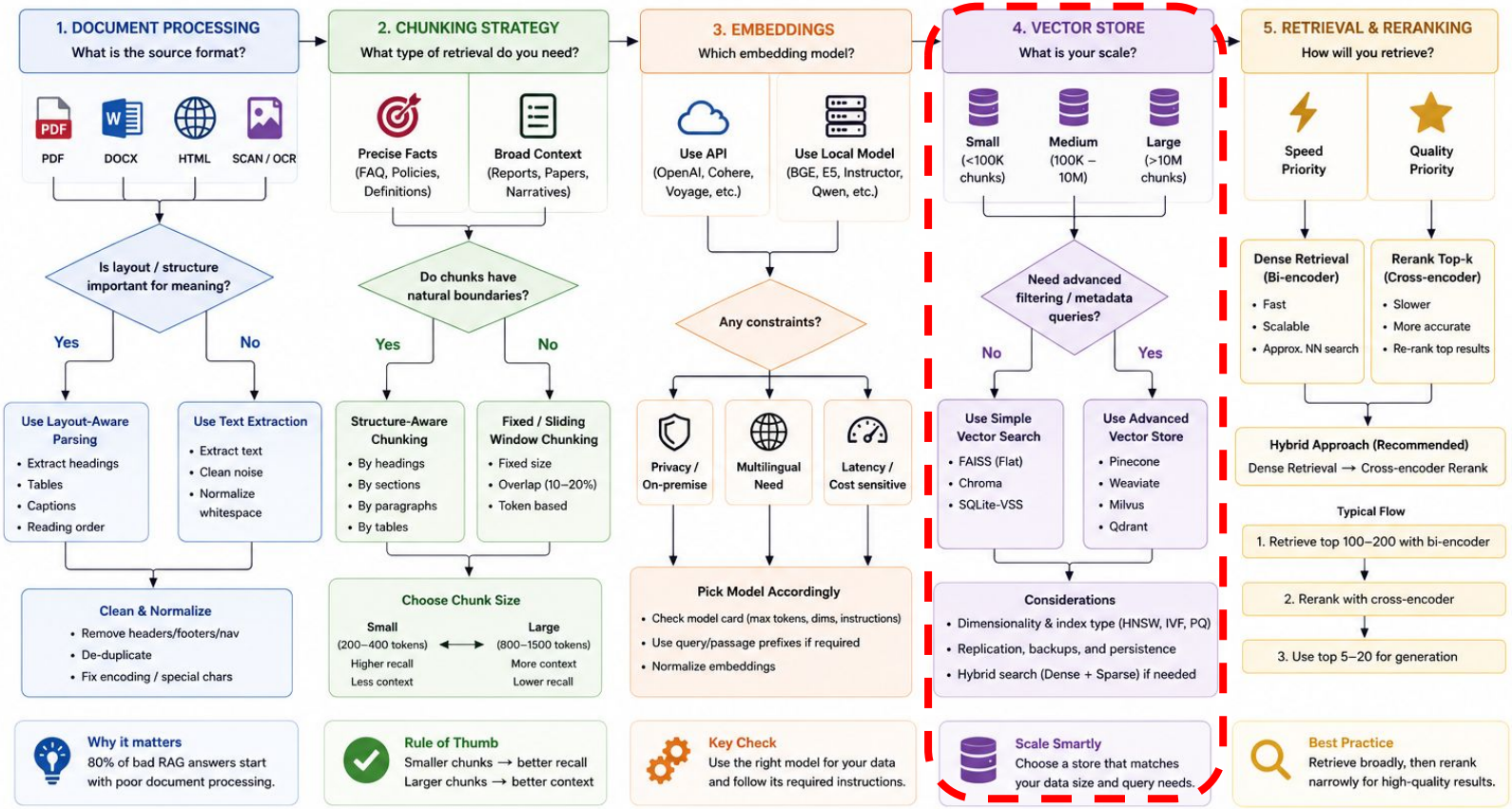
Highly Predictable: Follows narrow vocabulary and repetitive sentence length. Statistically Homogeneous: Lacks the varied sentence structures, chaos, and distinct stylistic fingerprints of human writers.

Data condition	Chunker	Decided by
Uniform prose, baseline	Fixed-size (token)	Simplicity
General RAG prose	Sliding window ~20%	Boundary-spanning answers
Markdown / legal / code	Structure-aware	Natural seams + provenance
Mixed, quality-first	Semantic (+ cap)	Cut at meaning shifts
Chat / transcripts	By turn/speaker	The unit is an utterance
Dense reference QA	Small-to-big	Precision + context
Tables / CSV	Row / schema-aware	Don't split records

Measure it: chunking is the easiest choice to A/B — swap the chunker in the eval harness and re-run. The chunk-size knob alone often moves retrieval failure more than a model swap.

Design Choices in RAG Systems

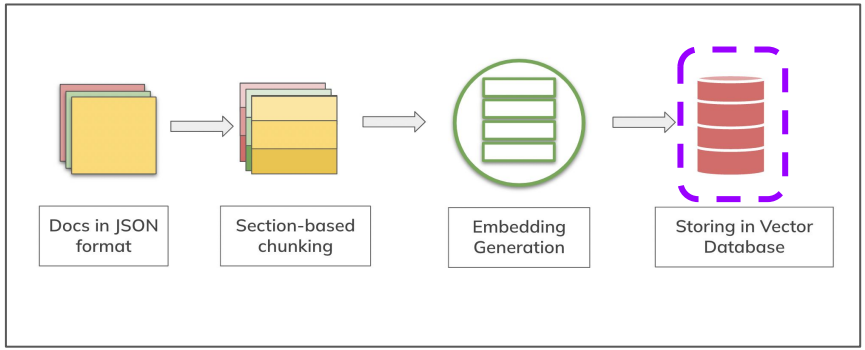
Documents, Chunking & Vector Stores



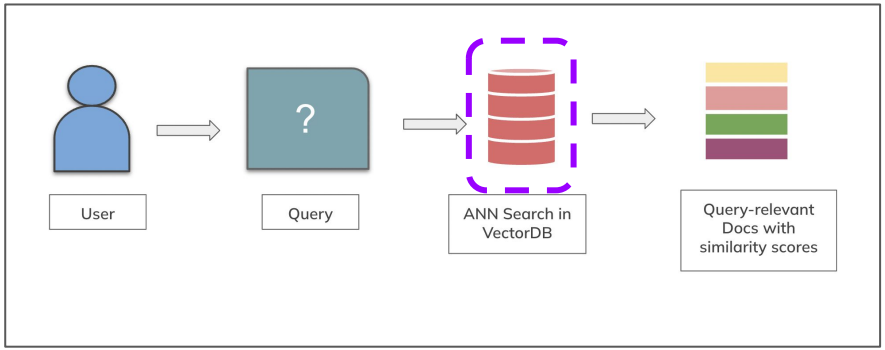
Key Principle Good RAG is about making the right design choices at every step — clean data, meaningful chunks, appropriate embeddings, scalable storage, and effective retrieval.

RAG with Vector DB

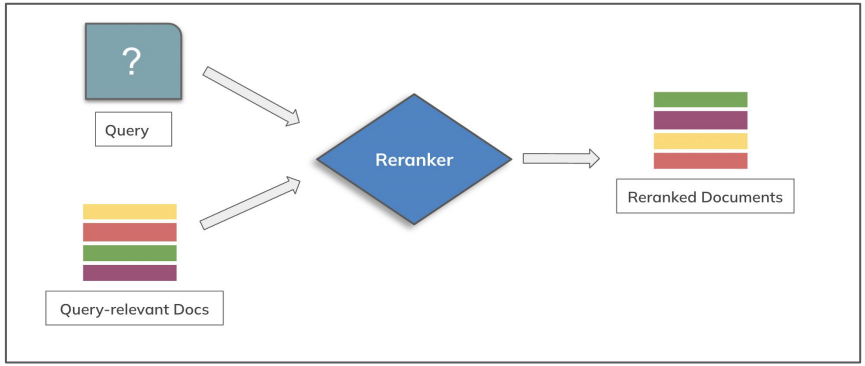
Pre-Retrieval



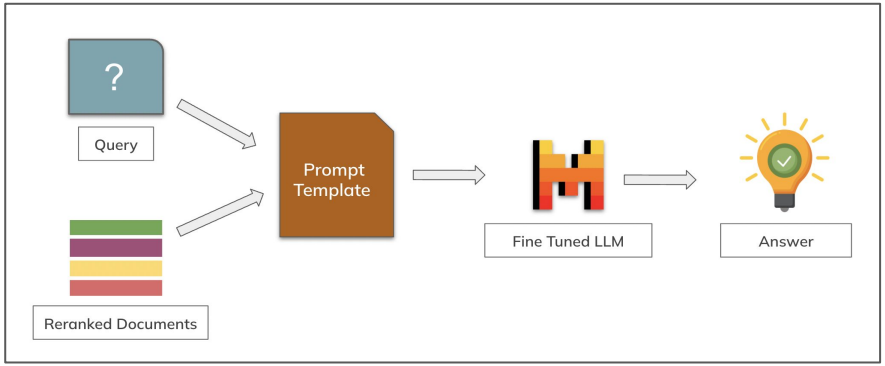
Retrieval



Post-Retrieval



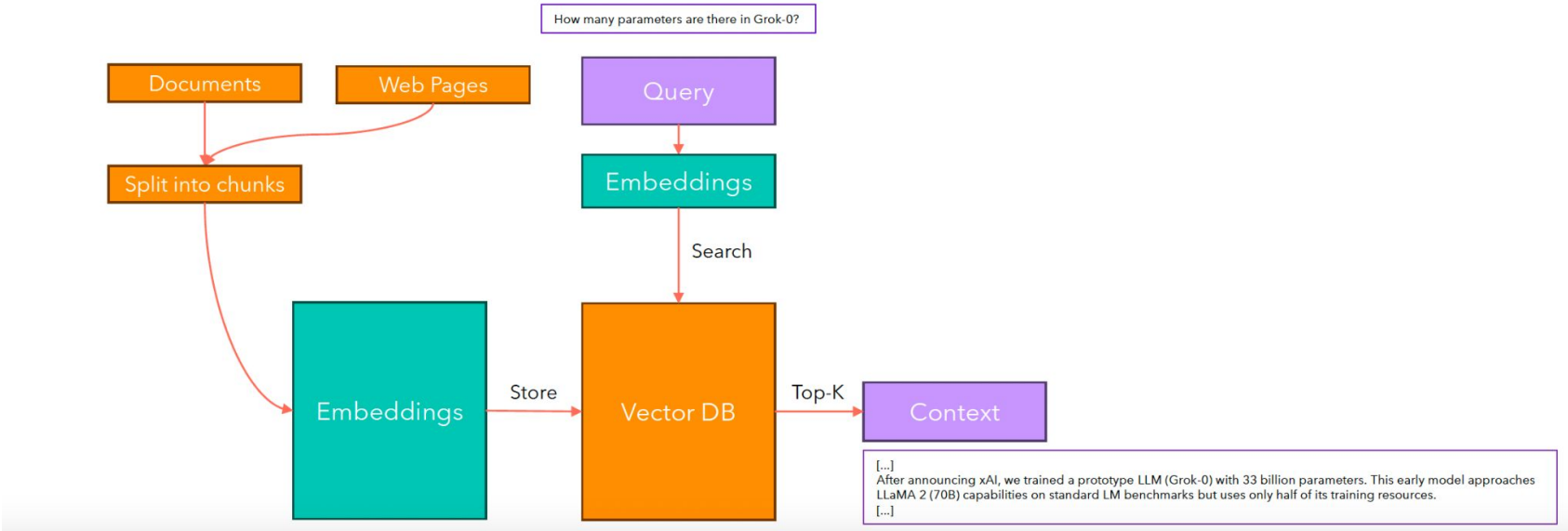
Augmentation and Generation



https://ai.meta.com/blog/retrieval-augmented-generation-streamlining-the-creation-of-intelligent-natural-language-processing-models/

Vector Database: Introduction

A vector database stores vectors of fixed dimensions (called embeddings) such that we can then query the database to find all the embeddings that are closest (most similar) to a given query vector using a distance metric, which is usually the cosine similarity, but we can also use the Euclidean distance. The database uses a variant of the KNN (K Nearest Neighbor) algorithm or another similarity search algorithm. Vector DBs are also used for finding similar songs (e.g. Spotify), images (e.g. Google Images) or products (e.g. Amazon).



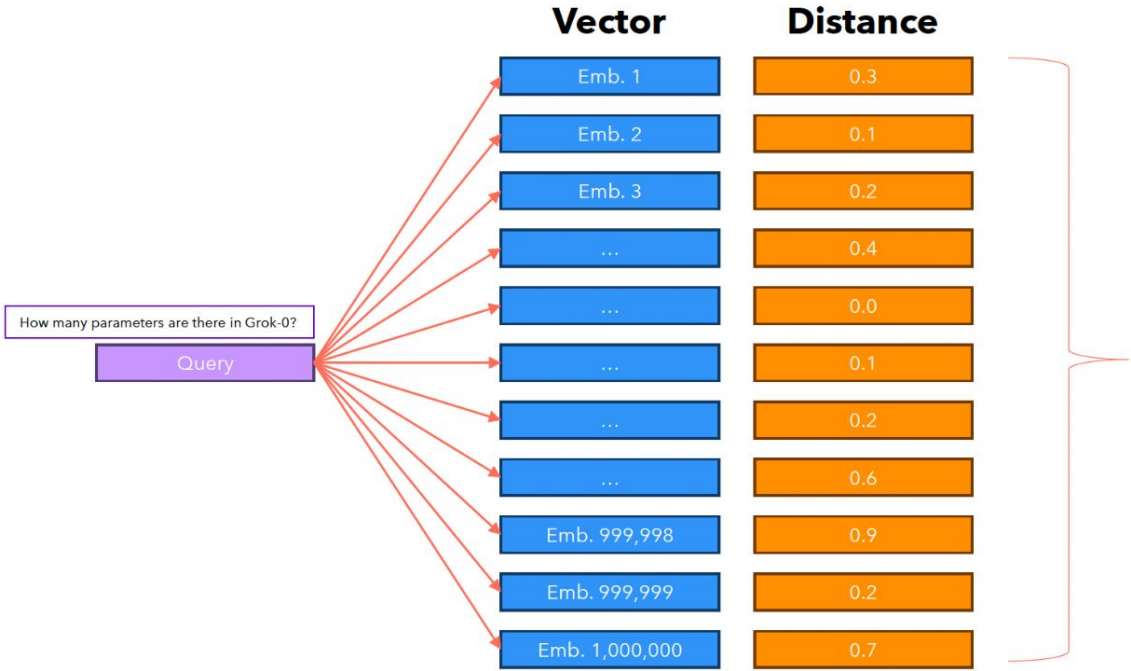
Source: https://github.com/hkproj/retrieval-augmented-generation-notes

Why Efficient Vector DB is required for RAG?

1. A critical aspect that powers the capabilities of Retrieval-Augmented Generation models is the vector database that stores the embeddings for fast semantic search during the initial retrieval stage.
2. In order for RAG models to scale to immense corpora containing billions of text passages, efficiently indexing and querying vector representations is crucial.
3. Examples - Weaviate, Chroma, FAISS, Vespa, or Pinecone

KNN: A naive approach

Imagine we want to search for the query in our database: a simple way would be comparing the query with all the vectors, sorting them by distance, and keeping the top K.

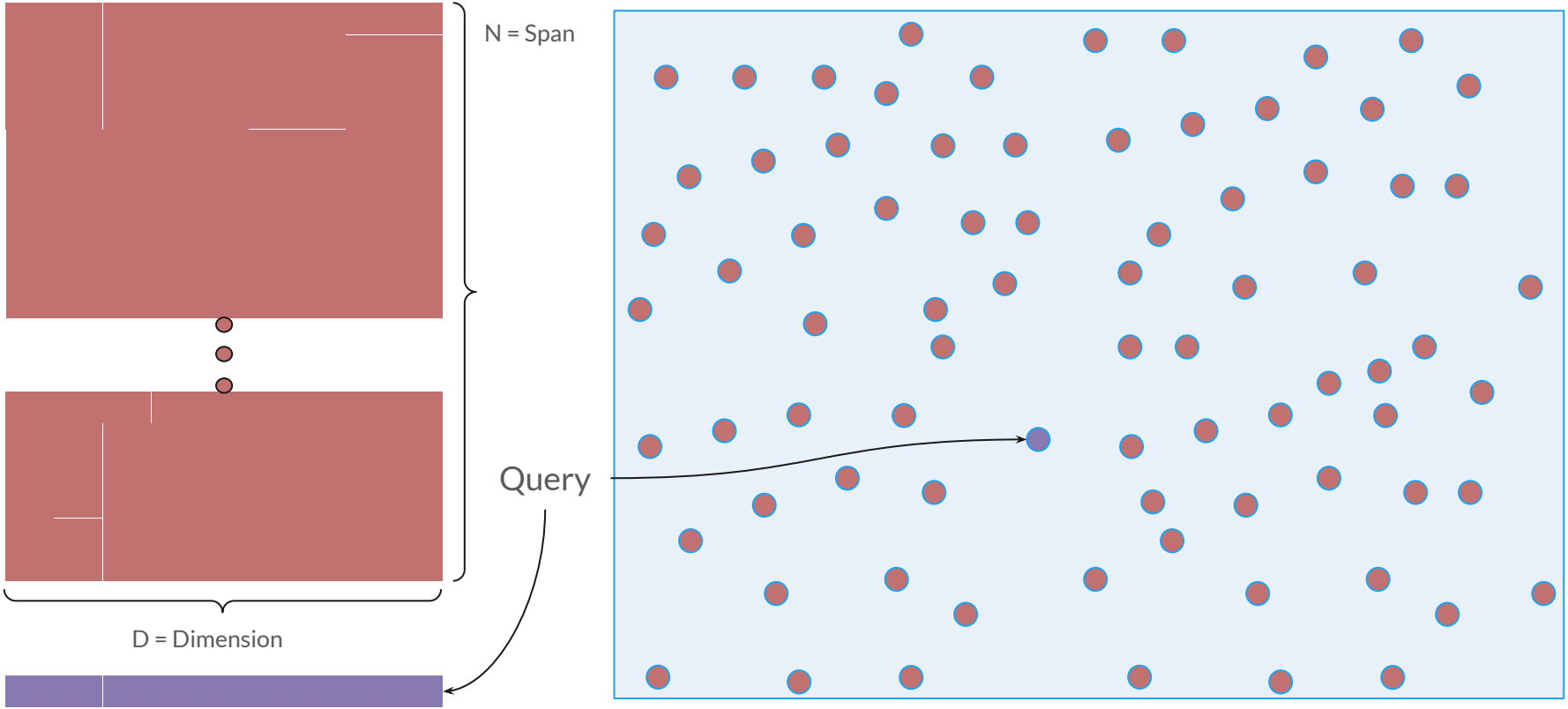




If there are N embedding vectors and each has D dimensions, the computational complexity is in the order of **O(N*D), too slow!**

Source: https://github.com/hkproj/retrieval-augmented-generation-notes

IVF: Inverted file for vector Database

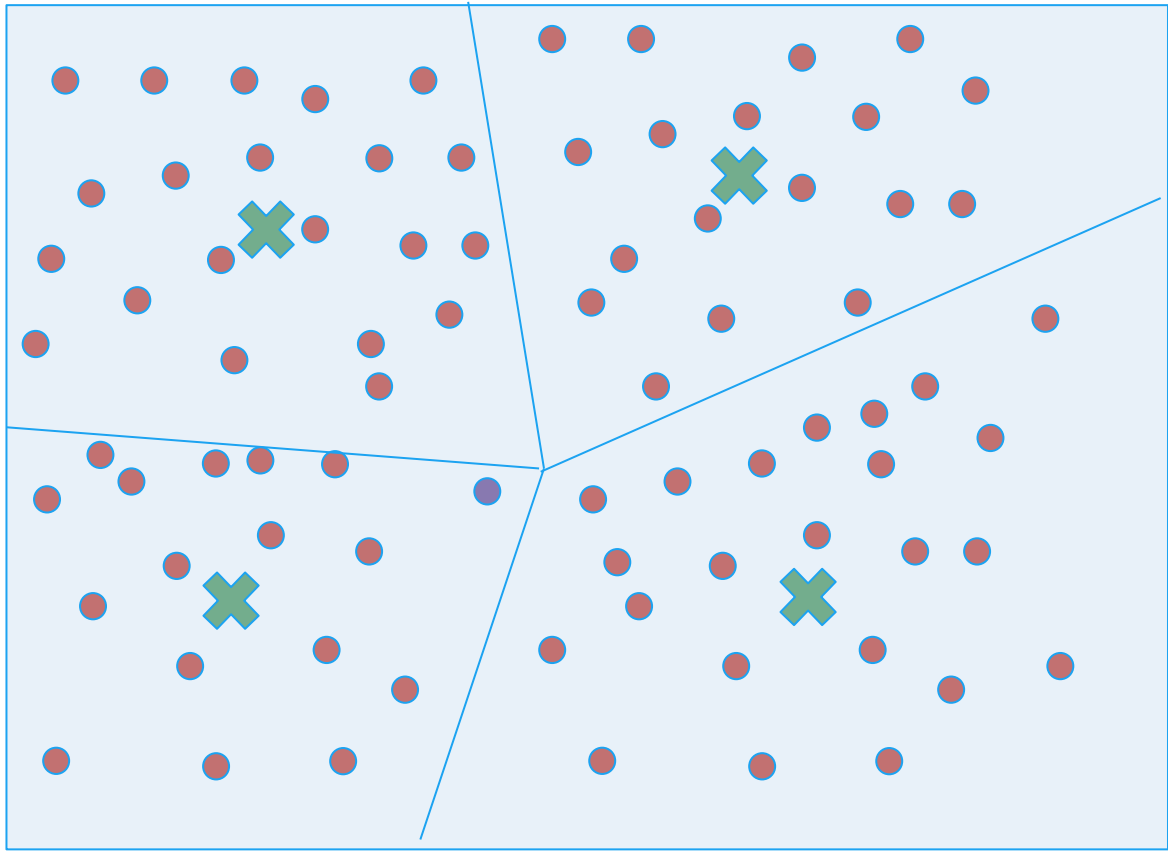


https://www.youtube.com/watch?v=BMYBwbkbVec&list=PLIUOU7oqGTLhlWpTz4NnuT3FekoulVlqc&index=9

IVF: Inverted file for vector Database

Phase 1: Indexing

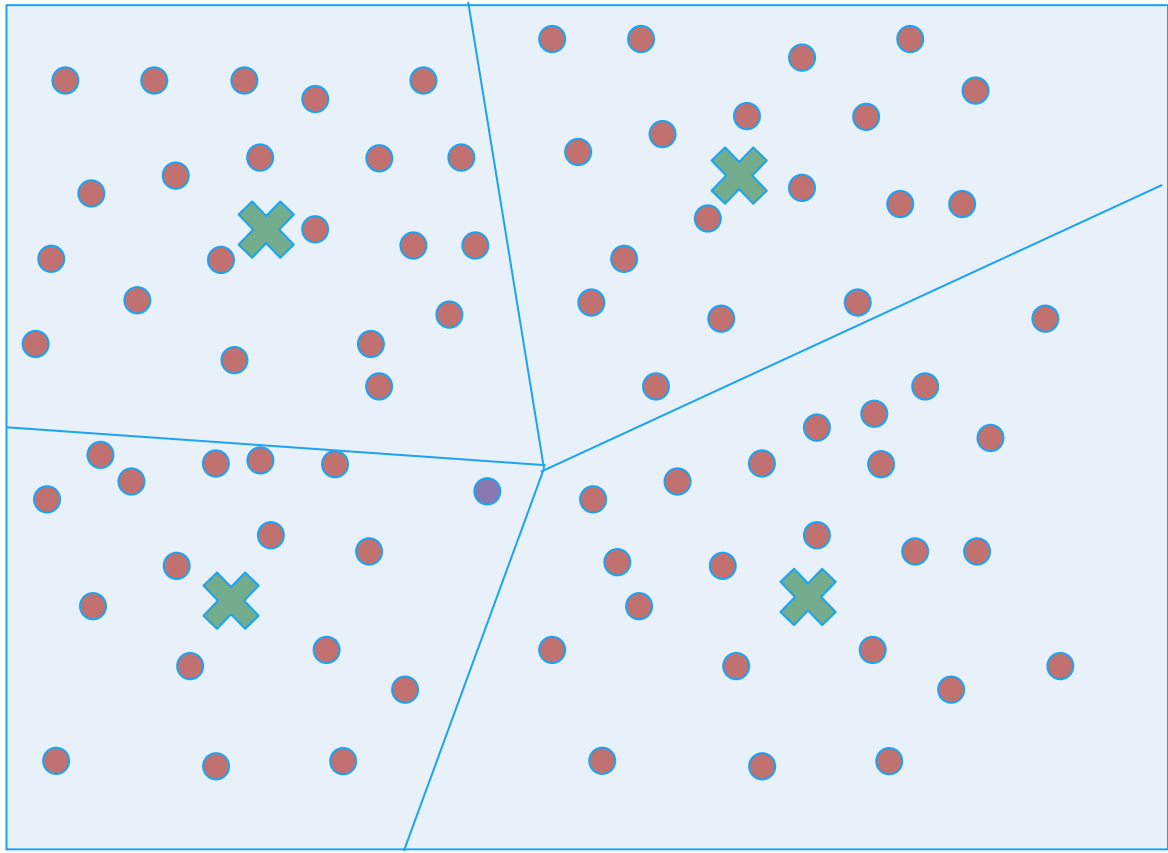
- **Train:** Run **K-Means** on the dataset to identify k representative centroids.
- **Partition:** Assign every vector to its nearest centroid (geometrically forming **Voronoi cells**).
- **Store:** Organize vectors into "buckets" (Inverted Lists) labeled by their centroid ID.



IVF: Inverted file for vector Database

Phase 2: Retrieval (The Search)

- **Coarse Search:** Compare the query **only** against the k centroids to find the closest match.
- **Fine Search:** Scan **only** the vectors inside that specific bucket (ignoring the rest)
- **nprobe** : #clusters (centroids) the algorithm evaluates during an ANN search (*=1 by default*)
- **Outcome:** Reduces search scope from O(N) to approximately O(N/k)



IVF: how much work do you actually save?

QUESTION

N = 1,000,000 vectors, nlist = √N buckets, nprobe = 1. Roughly how many vectors get scanned vs. brute force? Then: what does nprobe = 10 cost?

ANSWER

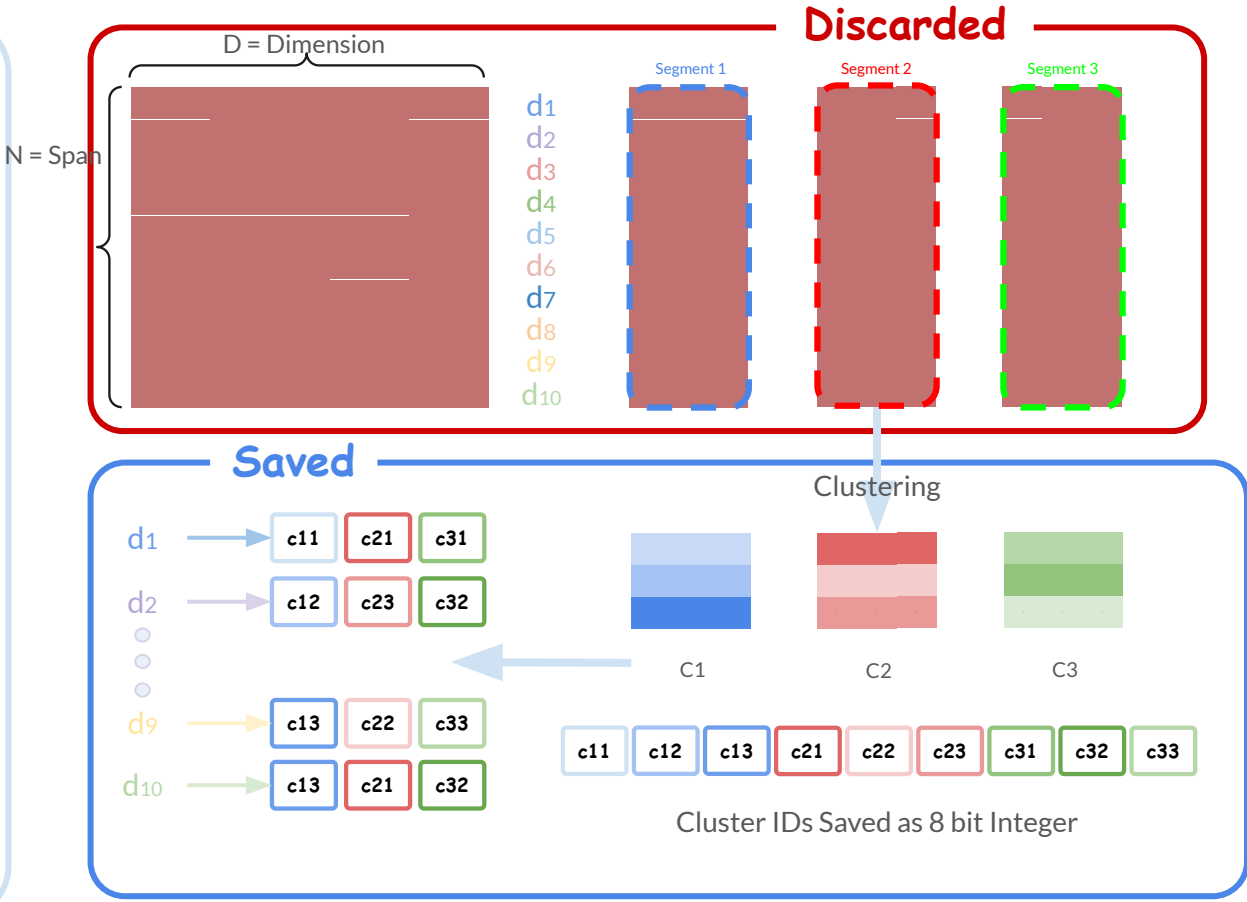
nlist = 1000 centroids + ~1000 vectors in the one bucket ≈ 2,000 comparisons vs. 1,000,000 — about a 500× reduction.

nprobe = 10 ⟶ 1000 + ~10,000 ≈ 11,000: still ~90× faster, with markedly higher recall.

PQ: Product Quantisation (Indexing)

Indexing (Compression Phase)

- **Split (Sub-vectors):** Divide the high-dimensional original vector (e.g., 128D) into m smaller sub-vectors (e.g., 8 chunks of 16D).
- **Train (Codebooks):** Run K-Means independently on each sub-space to find K (eg. 256) representative centroids (the "Codebook").
- **Encode:** Replace each sub-vector with the **ID** (0-255) of its nearest centroid.
- **Result:** High-precision floats are converted into small byte codes, reducing memory usage by 95%+ (e.g., 4KB →64 bytes).



PQ: The 95% memory claim

QUESTION

A 768-dim float32 vector is split into m = 8 sub-vectors, each encoded to one of 256 centroids. What is the stored size per vector, and the compression ratio?

ANSWER

Original: 768 × 4 bytes = 3,072 bytes. PQ code: 256 centroids → 8 bits = 1 byte each × 8 = 8 bytes.
3,072 / 8 = 384× smaller — i.e. a ~99.7% reduction. The "95%+" claim on the slide is comfortably met.

PQ search: compute once, then just add

The database vectors are stored as **centroid IDs**, not floats. So distance becomes a **table lookup + addition** — no float math per vector.

1 Once per query: build the distance table

Split the query into m sub-vectors. For each, measure its distance to that sub-space's centroids.

	c1	c2	c3	c4
q_1	0.9	**0.2**	0.7	0.5
q_2	0.4	0.6	**0.1**	0.8
q_3	0.3	0.7	0.9	**0.2**

m × k table — tiny, reused for every vector

2 Per database vector: look up + add

A stored vector is just its centroid IDs, e.g. **[c2, c3, c4]**. Read those cells and sum them.

$q_1 \to c2$ **0.2** + $q_2 \to c3$ **0.1** + $q_3 \to c4$ **0.2** = **0.5**

approx. distance — 3 lookups, 2 adds

Why it's fast — Asymmetric Distance Computation (ADC)

"Asymmetric" = the query stays full-precision while documents are compressed. The costly query–centroid math happens **once**; scanning a million vectors is then just lookups and additions — no per-vector float distance.

Contrast with the indexing slide: there you built the codebooks; here you reuse them to score fast.

Quiz 1

Q. A team's RAG recall is mediocre. They swap their E5 embedding model for a fancier one and recall barely improves. What's the most likely real cause?

a. Their chunks are too large
b. They forgot the query: / passage: prefix the asymmetric model expects
c. They're using Euclidean distance

Option b is correct . E5 models are asymmetric they're trained with explicit prefixes to distinguish query-side from document-side encoding. Without them, both query and passage vectors are projected into the same undifferentiated space, producing systematically misaligned representations

Euclidean vs. cosine distance matters mainly when embeddings aren't normalized — many modern pipelines normalize by default

Chunks being too large would show up as inconsistent recall across query types and would improve with a better model's longer context handling.

Quiz 2

Q. A team builds RAG for **PDFs with tables and multi-column layouts**. Accuracy is poor even though embedding model is strong. What is the most likely?

a. The cosine similarity is incorrect
b. The document parsing / chunking step is losing structure
c. The vector store is too small
d. The LLM context window is too large

Answer: 2 Document parsing/chunking is losing structure.

PDFs are notoriously hostile to naive text extraction. Multi-column layouts get linearized incorrectly (columns merged left-to-right across the page instead of read top-to-bottom per column), and tables get flattened into garbled token sequences that destroy row/column relationships entirely.

The fix lives in the parsing layer tools like pymupdf, pdfplumber, or dedicated document AI (e.g. AWS Textract, Azure Document Intelligence) that can handle layout-aware extraction and table serialization before chunking.

Quiz 3

3. N = 5,000,000 vectors, nlist = √N buckets, nprobe = 2. Roughly how many vectors get scanned vs. brute force? Then: what does nprobe = 10 cost?

a. 2,10

B. 6708, 24596 **(Correct Answer)**

C. 2048, 8192

d. 5000000, 500000

nlist = √5,000,000 ≈ 2,236 centroids, bucket size ≈ 2,236

nprobe = 2: 2,236 + 2×2,236 = **6,708** vectors

nprobe = 10: 2,236 + 10×2,236 = **24,596** vectors

Quiz 4

4. A 768-dim float32 vector is split into m = 8 sub-vectors, each encoded to one of 256 centroids. What is the size per vector and the compression ratio?

a. 8, 384
b. 64, 48
c. 32, 96
d. 256, 12

Sub-vector dimension: 768 / 8 = 96 dims each

Encoding: each sub-vector is mapped to 1 of 256 centroids, so you need $\log_2(256)$ = **8 bits** = **1 byte** to store the centroid index.

Size per vector:

	Calculation	Size
Original (float32)	768 × 4 bytes	3,072 bytes
PQ-compressed	8 × 1 byte	8 bytes

Compression ratio: 384×

3,072 / 8 = 384

Thank You

Instructor



Pawan Goyal

Teaching Assistants (IITKGP)



Pretam Ray



Kavin RV



Kunal Kingkar